Assignment 2: Deep Learning (Spring 2026)

The "Quick, Draw!" Challenge: MLP Architecture Analysis

Roll Number: 25280030

March 3, 2026

GitHub Repository

Link: `https://github.com/rana-taqveem/ai600/tree/main/assignment_02`

1 Part A: The Pancake (Width Focus)

Hypothesis: Does raw memorization power solve the problem?

As per the results of multiple wide model executions, we can see that the raw memorization power is insufficient, and failed to perform on unseen data. The **WideMLP** utilized **1,937,767 parameters**. While it achieved a peak training accuracy of **99.10%**, however its validation accuracy remained only at **76.67%**.

Performance Analysis and Visualizations

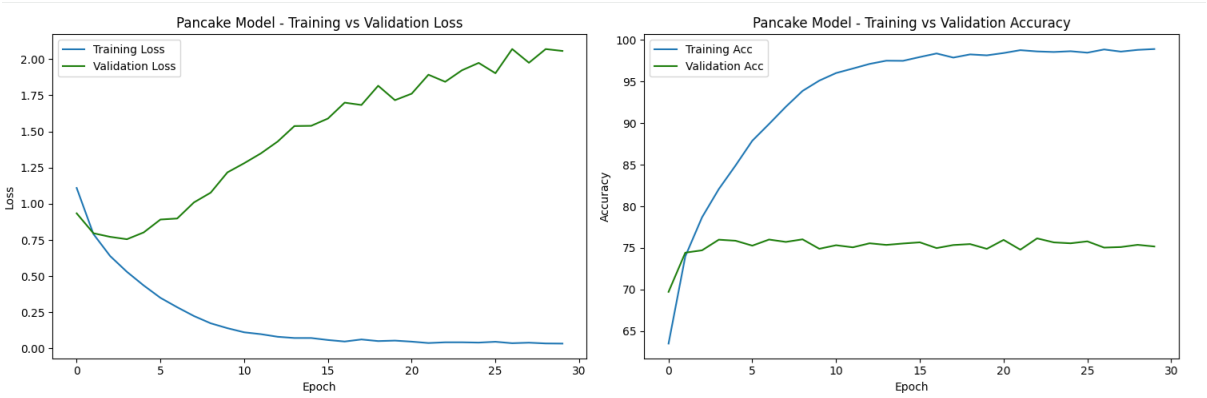

Figure 1: Training vs. Validation curves for the Wide (Pancake) MLP. The divergence starting at Epoch 5 indicates severe overfitting.

This clearly shows that the model failed to generalize, because it does not have the required structural depth to learn spatial features like edges and corners and the variations among them. Instead, it just memorised the mapping of raw pixel intensities. Consequently, while it worked exceptionally well on training data, it failed miserably on the unseen data

validation data because it could not recognize intra-class variations—such as a "Pizza" drawn with different slice counts or orientations.

2 Part B: The Tower (Depth Focus)

Hypothesis: Can a hierarchical structure learn better features with fewer parameters?

Yes. The Tower model, using only **833K parameters**, achieved a much higher validation accuracy of **86.5%**. By utilizing 5+ layers, this network managed to learn complex spatial features and edges and also variations in these features from shape to shape, which is fundamentally more efficient for high-dimensional data like images, thus resulting in performing substantially better than the Pancake model.

3 Part D: Theoretical Analysis

3.1 Width vs. Depth and the Universal Approximation Theorem

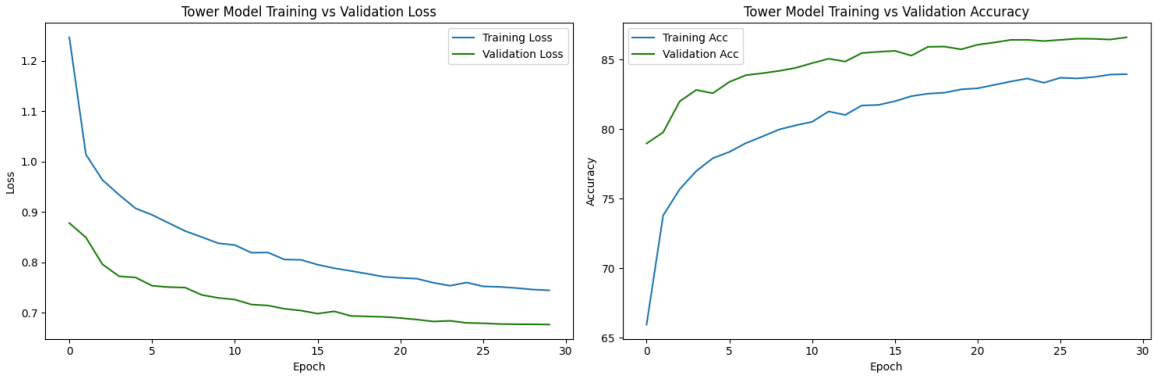

Figure 2: Training vs. Validation curves for the Deep (Tower) MLP, showing much better generalization compared to the Wide model.

While the **UAT** states a single hidden layer can approximate any continuous function, based on the above finding its rational to prefer a deep model for image based tasks because:

- **Feature Composition:** Depth allows the model to learn a hierarchy (edges ¿ strokes ¿ objects with different variations), which are mush closer to the structure of visual data.
- **Parameter Efficiency:** This allowed the deep model to achieve higher accuracy with fewer total parameters by reusing the learned features across layers.

3.2 Confusion Analysis

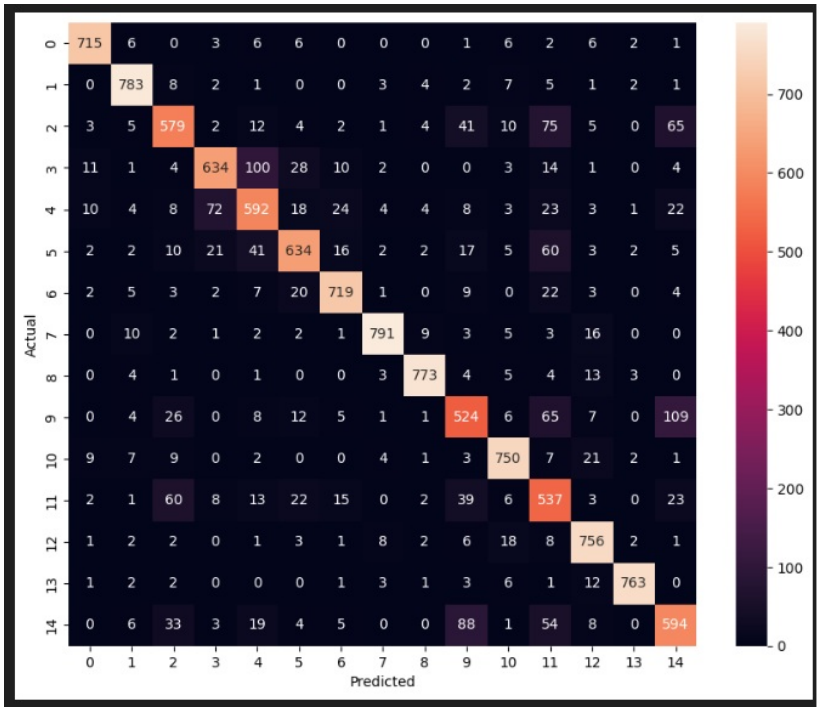

Figure 3: Confusion Matrix for the Champion Model.

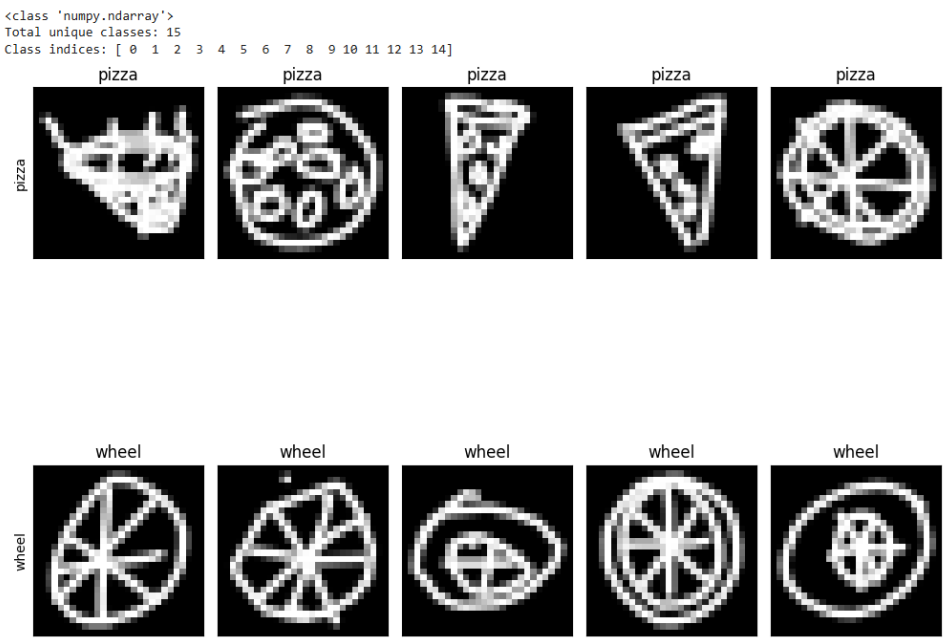

Figure 4: Visual samples of Class 9 (Pizza) and Class 14 (Wheel).

- **Top Confused Pair (Pizza vs. Wheel):** 109 instances of **Pizza** (Index 9) were predicted as **Wheel** (Index 14).
- **Rationale:** This confusion stems from the data ambiguity(aleatoric uncertainty) i.e the model fails to distinguish a circle if its a Pizza or a Wheel as both classes

share a circle as a primary stroke. Without clear internal textures and edges, even the augmented image with HOG feature vector failed because it lacks the spatial context to distinguish them.

4 Part E: Rationale and Summary

The **Champion Model** (553,839 parameters) utilized **Batch Normalization** and **Weight Decay** and minor dropout as well to ensure generalization well within the 3-million parameter limit.

4.1 Specific Depth and Width

- **Initial Feature Extraction (Layer 1 & 2):** The input vector of 784 pixels plus the HOG features (∼1200 total), the first two layers (L1=224 neurons, L2=192 neurons) play the role of the main information collection hub. These two layers capture the primary spatial information like basic edges, strokes and gradient information provided by HOG descriptors.
- **Compression and Refinement (Layer 3 & 4):** The next 2 hidden layers with 128 neurons each work as compression and refinement layers. The narrow width forces the network to discard the noise and focus on the important pieces of information like specific curvature or corners.
- **Convergence:** The final layer converges to 15 classes in the dataset to produce the predicted results based on what it learned from the previous layers.

This Tower structure helped in building a model that generalized better on unseen data as compared to the Pancake model, by forcing to learn abstract shapes (edges, strokes) instead of just memorizing pixels.

4.2 Regularization Strategy

To ensure the model prioritized generalization over raw memorization, I implemented a multi-faceted regularization strategy:

- **Batch Normalization:** As I am using mini-batch gradient descent where each batch has slightly different statistics (μ, σ^2), I added a Batch Normalization layer between the linear and non-linear layers to standardize activations to zero mean and unit variance. Since these batch-specific statistics differ from batch to batch, it introduces a small amount of noise into the activations, thus introducing a regularization effect which helps reduce overfitting.
- **L2 Regularization (Weight Decay):** I applied a weight decay of 5e-05 to penalize large weight values, preventing the "exploding weights" phenomenon and ensuring the model learned smoother decision boundaries and generalizes better.
- **Dropout:** Added dropout to randomly deactivate neurons during training, forcing the network to learn redundant feature paths. This prevented the model from relying too much on a few specific neurons, thus forcing the entire network to share the workload of better identifying doodles like the **Pizza** or **Wheel**.

As shown in the comparison curves (Figure 2), these techniques allowed the **Champion Model** to maintain a narrow gap between training and validation accuracy, whereas the unregularized **Pancake Model** diverged significantly.

Table 1: Model Comparison Summary

Metric	**Pancake (Wide)**	**Tower (Deep)**	**Champion**
Parameter Count	1,937,767	833,000	**553,839**
Training Epochs	30	30	**30**
Final Val Accuracy	75.22%	86.50%	**86.67%**

5 AI Usage

- **GitHub Copilot** was used for code assistance during model implementation.
- **AI tools** were used to assist in generating this LaTeX report.
- AI was used to understand topics and tradeoffs between techniques such as width vs. depth, regularization strategies, and feature extraction methods.